# ActuaryGPT: Agentic AI-Driven Insurance Underwriting & Claims Triage System

**Internship Project Technical Documentation & Systems Report**

*Prepared by: Ahan Mullick*

*Date: June 21, 2026*

---

## ■ Executive Summary

In the modern insurance landscape, underwriting and claims processing represent significant bottlenecks characterized by manual document review, complex risk evaluation, and susceptibility to fraud. **ActuaryGPT** is a next-generation, web-based software suite that automates health risk profiling and vehicle claims triage.

By integrating **CatBoost Machine Learning models** for risk and fraud probability prediction, **Retrieval-Augmented Generation (RAG)** via cosine similarity indexing for historical case lookup, and the **Google Gemini Large Language Model (LLM)** for generating natural language actuarial justifications, ActuaryGPT accelerates underwriting timelines from days to seconds while improving decision auditability.

---

## ■ Problem Statement & Business Objectives

Legacy core insurance platforms suffer from:

1. **Inefficient Underwriting Triage:** Underwriters manually inspect health history records, medical reports, and applicant metrics to classify policy pricing, creating high operational overhead.
2. **High Fraud Exposure:** Vehicle claims are often filed without rigorous verification of historical similarities, leading to duplicate payouts or unflagged fraudulent collusion.
3. **Lack of Explainability:** Traditional ML predictions act as "black boxes," leaving underwriters without clear context on why an applicant was classified as high-risk.
4. **Segmented Customer Communication:** Policyholders lack transparent tracking of their claims and secure chat channels to address underwriting queries.

### ActuaryGPT Core Objectives

- **Automation:** Complete risk profiling of life policy applications and vehicle claims automatically in under 5 seconds.
- **Accuracy:** Use CatBoost models trained on actuarial data to classify health risk profiles and detect claim anomalies.
- **Explainability:** Generate natural language reasoning summaries (Underwriter Briefs) matching regulatory compliance.

- **Ledger Auditing:** Retrieve top similar cases from the historical reference database using vector similarity search to prevent duplicate claims.
- **Secure Access Controls:** Segment operations into a public customer-facing self-service wizard and a secure, officer-restricted risk command dashboard.

---

## ■■ System Architecture & Data Flows

The platform is designed around a decoupled, service-oriented architecture:

```
graph TD
    subgraph Client Tier [React/Vite Application]
        A[Client Web Browser] -->|Routes| B{Role Router}
        B -->|Customer Role| C[Self-Service Portal]
        B -->|Officer Role| D[Underwriter Command Center]

        C -->|Form Input / OCR Upload| E[Smart Multi-Step Wizard]
        D -->|Triage Queue Grid| F[AI Profiler Dashboard]
        D -->|Analytics Tab| G[Dynamic Recharts Suite]
        D -->|Underwriter Co-Pilot| H[Actuarial Chatbox]
    end

    subgraph Service Tier [FastAPI REST API Server]
        I[FastAPI Application Instance]
        I -->|Endpoints| J[Authentication Router]
        I -->|Endpoints| K[Life Underwriting Router]
        I -->|Endpoints| L[Vehicle Claims Router]
        I -->|Endpoints| M[Analytics & Ledger Router]
        I -->|Endpoints| N[Orchestrator Chatbot]
    end

    subgraph Database Tier [SQLite Reference Store]
        O[(actuary_gpt.db)]
        O -->|Contains| P[users Table]
```

```
    O -->|Contains| Q[applications Table]
    O -->|Contains| R[vehicle_applications Table]
end

subgraph AI Pipeline [Automated Auditing Engine]
    S[Document Processing OCR] --> T[CatBoost Classification Model]
    T --> U[Jaccard & Cosine RAG Matcher]
    U --> V[Gemini Explanation Agent]
    V --> W[ReportLab PDF Compiler]
end

E -->|JSON Payloads| I
F -->|Trigger Pipeline| I
G -->|Get Metrics| I
H -->|Query| I
I -->|Read/Write| O
K -->|Triggers| AI Pipeline
L -->|Triggers| AI Pipeline
W -->|Save Reports| X[Static File System]
```

### End-to-End Evaluation Pipeline Dataflow

1. **Data Ingestion:** The applicant submits form metrics (such as age, BMI, family history) or files a claim document (OCR scans medical bills/vehicle repair sheets).
2. **ML Classification:** The backend preprocesses the features and passes them to the CatBoost risk model:

- **Life Insurance:** Predicts a risk class from $1$ (lowest risk) to $8$ (highest risk).
- **Vehicle Insurance:** Predicts fraud risk probability ($0\%$ to $100\%$).

3. **RAG Vector Search:** The database is queried for historical matches using a hybrid cosine-similarity algorithm.
4. **Co-pilot Summarization:** The features, ML predictions, and top matching cases are structured into a prompt context and sent to Google Gemini (e.g., `gemini-2.5-flash`). The LLM returns a comprehensive Actuarial Brief detailing risk factors and recommendations.
5. **Ledger Update:** The application state is saved to the SQLite database, and a PDF document is generated for download.

---

# ■ Core Algorithmic Details & ML Pipelines

## 1. Life & Health Risk Predictor

The Life model assesses health metrics to classify applicants into risk classes ($1-8$).

- **Input Features:** `Age`, `Height`, `Weight`, `BMI`, `Smoker` status, `Previous Claims`, `Family History`, `Occupation`, `Income`, `Exercise` frequency, and `Alcohol` usage.
- **Pre-processing:** Categorical features (such as occupation or gender) are pre-mapped using integer encoders or standard labels:

```
 # Category mapping dictionary example

 "D2" = 12.0, "A1" = 1.0, "E1" = 18.0
```

- **ML Model:** CatBoost classifier outputting risk category probabilities. A lower class represents standard underwriting (lower premium rates), while classes $6-8$ are flagged as high risk (referred for manual review or declined standard rates).

## 2. Vehicle Claims Fraud Classifier

The Vehicle model evaluates the probability of fraud based on incident profiles.

- **Input Features:** `months_as_customer`, `policy_annual_premium`, `policy_deductable`, `insured_sex`, `insured_occupation`, `insured_relationship`, `capital_gains`, `capital_loss`, `incident_type`, `collision_type`, `incident_severity`, `property_damage`, `total_claim_amount`, `auto_make`, `auto_model`, `auto_year`, and `witnesses`.
- **Model:** CatBoost model mapping tabular variables to classify high-risk claims based on historical claims data.

## 3. Cosine Similarity RAG Retrieval

To prevent duplicate payout claims and identify systemic fraud rings, ActuaryGPT runs a vector-based search over historical databases using cosine similarity:

$$\text{Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \sqrt{\sum_{i=1}^{n} B_i^2}}$$

#### Python Implementation (`similarity.py`):

The search engine extracts numeric dimensions, normalizes the feature vectors, and computes similarities to retrieve matching histories:

```
def calculate_life_similarity(app1, app2):

    # Features compared: age, height, weight, bmi, income, coverage

    v1 = [app1['age'], app1['height'], app1['weight'], app1['bmi'], app1['income'], app1['c
overage_amount']]

    v2 = [app2['age'], app2['height'], app2['weight'], app2['bmi'], app2['income'], app2['c
overage_amount']]



    dot_product = sum(a * b for a, b in zip(v1, v2))
```

```
    magnitude_1 = sum(a*a for a in v1) ** 0.5
    magnitude_2 = sum(b*b for b in v2) ** 0.5

    if magnitude_1 == 0 or magnitude_2 == 0:
        return 0.0
    return dot_product / (magnitude_1 * magnitude_2)
```

---

## ■■ Database Schema Specification

The application uses an SQLite database `actuary_gpt.db` structured with three core relational tables:

```
Table: users
+------------+---------+-----------------------------------------------+
| Column     | Type    | Constraints                                   |
+------------+---------+-----------------------------------------------+
| id         | INTEGER | PRIMARY KEY AUTOINCREMENT                     |
| username   | TEXT    | UNIQUE, NOT NULL                              |
| password   | TEXT    | NOT NULL                                      |
| role       | TEXT    | NOT NULL, CHECK(role IN ('customer','officer')) |
+------------+---------+-----------------------------------------------+

Table: applications (Life & Health Policies)
+----------------------+---------+-----------------------------------+
| Column               | Type    | Constraints                       |
+----------------------+---------+-----------------------------------+
| id                   | TEXT    | PRIMARY KEY                       |
| client               | TEXT    | FOREIGN KEY REFERENCES users      |
| age, height, weight  | REAL    | NOT NULL                          |
| bmi                  | REAL    | NOT NULL                          |
| product_info_2       | TEXT    | NOT NULL                          |
| occupation, income   | REAL    | NOT NULL                          |
```

```
| smoker, previous_claims| INTEGER | NOT NULL                            |
| family_history        | INTEGER | NOT NULL                            |
| insurance_type        | TEXT    | NOT NULL                            |
| coverage_amount       | REAL    | NOT NULL                            |
| exercise, alcohol     | INTEGER | NOT NULL                            |
| gender, date          | TEXT    | NOT NULL                            |
| status                | TEXT    | CHECK IN ('pending','approved','rejected') |
| risk_class            | INTEGER | NULLABLE                            |
| risk_category         | TEXT    | NULLABLE                            |
| premium               | REAL    | NULLABLE                            |
| report                | TEXT    | NULLABLE (LLM Actuarial Summary)    |
| pdf_url               | TEXT    | NULLABLE                            |
+-----------------------+---------+-------------------------------------+

Table: vehicle_applications (Vehicle Claims)
+-----------------------+---------+-------------------------------------+
| Column                | Type    | Constraints                         |
+-----------------------+---------+-------------------------------------+
| id                    | TEXT    | PRIMARY KEY                         |
| client                | TEXT    | FOREIGN KEY REFERENCES users        |
| policy_state, policy_csl| TEXT  | NOT NULL                            |
| policy_deductable     | REAL    | NOT NULL                            |
| policy_annual_premium | REAL    | NOT NULL                            |
| insured_sex, occupation| TEXT   | NOT NULL                            |
| incident_severity     | TEXT    | NOT NULL                            |
| collision_type        | TEXT    | NOT NULL                            |
| total_claim_amount    | REAL    | NOT NULL                            |
| fraud_reported        | TEXT    | 'Y' or 'N'                          |
| risk_class            | INTEGER | NULLABLE                            |
| risk_category         | TEXT    | NULLABLE                            |
```

```
| confidence            | REAL    | NULLABLE                              |
| status                | TEXT    | CHECK IN ('pending','approved','rejected') |
+-----------------------+---------+---------------------------------------+
```

---

## ■ API Endpoints Reference

### 1. Authentication Router (`/auth`)

- `POST /auth/register`
- **Payload:** `{"username": "ahan", "password": "securepassword", "role": "customer"}`
- **Response:** `{"message": "Registration successful"}`
- `POST /auth/login`
- **Payload:** `{"username": "actuary1", "password": "password123"}`
- **Response:** `{"username": "actuary1", "role": "officer", "token": "..."}`

### 2. Underwriting Engine Router (`/applications`)

- `POST /applications`
- **Description:** Submits a life policy application.
- **Payload:** A complete JSON representation of applicant metrics.
- **Response:** `{"id": "APP-5261", "status": "pending"}`
- `POST /applications/{id}/evaluate`
- **Description:** Executes the AI pipeline (CatBoost models, similarity comparisons, Gemini reasoning).
- **Response:** Returns risk class ($1-8$), similarity array, and Markdown summary text.
- `POST /applications/{id}/decide`
- **Payload:** `{"decision": "approve" | "reject" | "manual_review", "modified_amount": 150000}`
- **Response:** Registers the state changes to the SQLite database.

### 3. Analytics Summarizer (`/analytics`)

- `GET /analytics/summary`
- **Description:** Returns dashboard stats (monthly claims, fraud distributions, premium totals).
- **Response:**

```
  {
    "total_policies": 999,
    "total_premiums": 6069411.61,
    "avg_risk_class": 4.5,
    "approval_rate": 99,
```

```
    "risk_distribution": {"High Risk": 90, "Low Risk": 63, "Medium Risk": 91},
    "processed_list": [...]
  }
```

---

## ■ UI/UX Design System Specifications

ActuaryGPT uses a dark, professional user interface to reduce eye strain for underwriters.

### 1. Core Visual Variables

```
:root {
  --bg-main: #0b0f19;         /* Very dark slate */
  --bg-card: rgba(17, 24, 39, 0.7); /* Translucent slate card */
  --primary: #6366f1;         /* Indigo accent */
  --secondary: #a855f7;       /* Violet accent */
  --border: rgba(255, 255, 255, 0.08);
  --text-title: #f3f4f6;      /* Off-white */
  --text-muted: #9ca3af;      /* Grey */
  --risk-low: #10b981;        /* Emerald green */
  --risk-medium: #f59e0b;     /* Amber */
  --risk-high: #ef4444;       /* Crimson */
}
```

### 2. Design System Components

- **Glassmorphism Cards:** Utilizes CSS `backdrop-filter: blur(12px)` and subtle borders to create deep layouts.
- **Animations:** Subtle CSS transitions (`transition: all 0.2s ease`) and keyframe fade-ins (`animation: fadeIn 0.4s ease-out`) for interactive actions.
- **Recharts Dashboards:** Custom tooltips and color maps display risk distribution and monthly volume trends.

---

## ■ Setup, Running & Build Guide

### Running locally

To start the development environment locally:

#### 1. Start backend server

```
cd backend

python -m venv venv

venv\Scripts\activate

pip install -r requirements.txt

python -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8000
```

#### 2. Start frontend dev server

```
cd frontend

npm install

npm run dev
```

Open `**http://localhost:5173/**` to access the dashboard locally.

### Cloud Deployment Strategy

- **Frontend:** Hosted on **Vercel** (`actuarygpt-5h5s.vercel.app`).
- **Backend:** Hosted on **Render** (`actuarygpt-backend.onrender.com`).
- **Cross-Origin Configuration (CORS):** The FastAPI instance uses `CORSMiddleware` with `allow_origins=["*"]` to ensure the deployed frontend can securely communicate with either the local backend or the Render host.
- **Secure Hybrid API Connection:** The application dynamically configures the target API host based on where it is loaded, preventing Mixed Content blocking by the browser:

```
 const API_BASE = window.location.hostname === "localhost" || window.location.hostname === "127.0.0.1"

   ? "http://127.0.0.1:8000"

   : "https://actuarygpt-backend.onrender.com";
```

---

## ■ Future Development Roadmap

1. **Multi-Tenant Roles:** Expand permissions to include auditors, reinsurers, and field agents.
2. **Blockchain Verification:** Anchor SHA-256 hashes of generated reports on a blockchain ledger for tamperproof records.
3. **Advanced ML Pipeline:** Upgrade CatBoost models using deep feature engineering to support custom commercial policies.
4. **Enhanced Chat Memory:** Add vector database storage (such as pinecone) to store co-pilot chat memories.